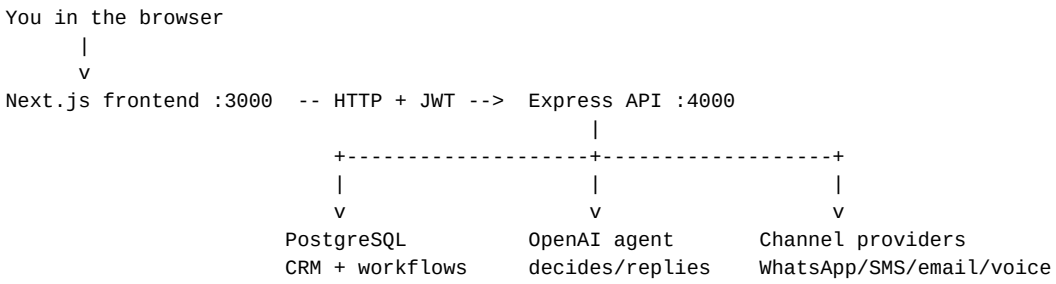
to check passwords, talk to the database, or use secret API keys. This a router for HTTP requests. Look at how `app.use("/api/auth", authRouter)` you write `prisma.contact.findMany({ where: { organizationId } })` and type errors if you get a field name wrong. Run `npx prisma studio` from touching the database. Never trust incoming data - validate first. the browser (fast, secure). Any component that needs interactivity -

Rohit AI Studio: Simple Project Guide

This guide shows how to run the project, use its features, and understand the path from a customer message to an AI response and a CRM update.

The whole project in one picture



The **frontend** is what the team uses. The **backend** checks permissions, talks to the database, runs agents/workflows, and keeps provider secrets private. PostgreSQL stores contacts, leads, conversations, messages, products, appointments, agents, workflows, and delivery history.

Start the project

You need Node.js, Docker Desktop, and the backend environment file.

1. Open Docker Desktop.
2. In Terminal 1, start the database and API:

```
cd "/Users/rohitmehra/Downloads/buisness ai/backend"
docker compose up -d
npm install
npx prisma migrate deploy
npm run dev
```

1. In Terminal 2, start the website:

```
cd "/Users/rohitmehra/Downloads/buisness ai/frontend"
npm install
npm run dev
```

1. Open http://localhost:3000, choose **Create a workspace**, and register your real business. Enter the email code to verify the address; the account is created only after verification. The current local database is empty; no demo data has been added.

Keep both `npm run dev` terminals open while you use the site. The API health check is http://localhost:4000/api/health.

Secure sign-in and email OTP

- **New workspace:** enter your name, business name, email, and password. The server sends a six-digit email code; verify it to create the account and workspace.
- **Password login:** enter your email and password, then verify the email code before a session is issued.

- **Passwordless login:** choose **Sign in with an email code**, enter your email, and verify the code.
- Codes expire in 10 minutes, can be resent after 30 seconds, and allow five attempts. The code is not returned by the API.
- With `NODE_ENV=development` and no Resend configuration, the code appears on the verification screen and in the backend terminal. Production requires Resend and never returns the OTP in an API response. Configure `NODE_ENV=production`, `OTP_SECRET`, `RESEND_API_KEY`, and `RESEND_FROM_EMAIL` for production email delivery.

What each feature does

Overview

Shows workspace activity, leads, conversations, upcoming AI bookings, and won pipeline value.

CRM and lead management

Add contacts, search the directory, create leads, and move leads between stages. Contact and lead data is stored in PostgreSQL and scoped to your business.

AI agents and conversations

Create an agent with a name, role, and instructions. Mark one as the default. The AI uses the active agent instructions together with your business profile and active offers. Conversations and messages are saved to the CRM. The inbox supports human takeover, replies, resolve, and reopen.

WhatsApp automation

Use **Simulate a message** to test the AI locally. For real WhatsApp, add a Meta access token to `backend/.env`, link the business Phone Number ID in Business Setup, and configure Meta's webhook to `/api/webhooks/whatsapp`. See WHATSAPP_SETUP.md for Meta setup.

SMS and email

The Channels & Calls page sends SMS through Twilio and email through Resend. Add provider credentials to the backend environment and a sender number/email in Business Setup. Each outbound message is recorded with a delivery status and provider ID.

AI telecalling

Choose a contact and agent on Channels & Calls to start an outbound Twilio voice call. The call needs Twilio credentials, a phone number, and a public HTTPS API URL so Twilio can reach the voice webhook. Speech turns and call status are saved in the CRM.

Business process automation

Create a workflow by choosing a trigger, optional condition, and action. Supported triggers are **Lead created**, **Customer message received**, and **Appointment booked**. Actions can send a channel message or update a lead stage. Use `{{contact.name}}`, `{{message.text}}`, `{{lead.status}}`, or `{{appointment.service}}` in message templates. Recent successes and failures appear in the execution log.

Appointments and offer library

The AI can check availability and book appointments during a conversation. The offer library is the AI's source of truth for prices and service details. Manual appointment creation, editing, and cancellation are not available yet.

Provider setup

Provider secrets belong only in `backend/.env`. Never paste keys into the browser or commit them to source control.

Feature	Backend environment variables	Business Setup
AI replies	`OPENAI_API_KEY`	Add business facts and agent instructions
WhatsApp	`WHATSAPP_ACCESS_TOKEN`, `WHATSAPP_VERIFY_TOKEN`	Meta Phone Number ID
SMS	`TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`	Twilio sender number
Voice	Twilio credentials and `PUBLIC_API_URL`	Twilio sender number; API must be publicly reachable
Email and OTP	`RESEND_API_KEY`, `RESEND_FROM_EMAIL`	Verified email sender

The Channels & Calls page reports **Needs setup** until the required settings are present. A successful API request is not claimed when the provider rejects a send.

How a customer message flows

1. WhatsApp, SMS, voice, or the simulator delivers an event to the backend.
2. The backend verifies the account/provider and identifies the organization and contact.
3. The message is stored in PostgreSQL and matching automation workflows run.
4. The selected AI agent reads business instructions, products, and conversation history.
5. The AI may check availability, book an appointment, update a lead, or escalate to a person.
6. The response is stored in the conversation and sent through the provider.
7. The inbox shows the conversation; message/call delivery state and automation runs can be inspected.

Important project files

File	What it controls
`frontend/app/page.tsx`	Main signed-in workspace and page state
`frontend/app/workspace-modules.tsx`	Agent, automation, and channel controls
`frontend/app/globals.css`	Application appearance and responsive layout
`backend/src/server.ts`	API routes, middleware, CORS, and error handling
`backend/src/routes/`	Authentication, CRM, channel, agent, workflow, and webhook APIs
`backend/src/services/orchestrator.ts`	AI prompt, tools, and customer-message flow
`backend/src/services/automation-engine.ts`	Workflow conditions, actions, and run records
`backend/src/services/outbound-messages.ts`	CRM message persistence and delivery tracking
`backend/src/lib/channel-providers.ts`	Twilio, Resend, and WhatsApp adapters
`backend/prisma/schema.prisma`	Database models and relationships
`backend/prisma/migrations/`	Versioned database changes

Safe development commands

```
# Run from backend/
npm run build
npm test
npx prisma migrate status

# Make a database change during development
npx prisma migrate dev --name describe-your-change
```

`npm run seed` creates a demo business and sample CRM data; it is optional. Do not run it if you want to keep the database real and empty until you register your own workspace.

When something fails

- **Website unavailable:** check the terminal running `frontend/npm run dev` and confirm port 3000.
- **Sign-in or save says "Load failed":** check `http://localhost:4000/api/health` and keep the backend terminal running.
- **Database/Prisma error:** run `npx prisma migrate status` from `backend/`; do not reset the database to fix a migration warning.
- **AI replies with a fallback:** verify `OPENAI_API_KEY` and agent/business instructions, then inspect the backend terminal.
- **Channel says "Needs setup":** check backend credentials and the matching sender settings. Voice webhooks also need a public HTTPS `PUBLIC_API_URL`.
- **Automation failed:** open Automation Engine -> Recent runs and read the recorded error.

Current limits

The project does not yet have a document knowledge base with embeddings, billing, or manual appointment create/edit/cancel controls. Live delivery requires your own approved provider accounts, verified senders, and credentials.